DSL for Table-Top Role-Playing Games (TTRPG)

in Python

S.Lott

https://fosstodon.org/@slott56
https://github.com/slott56

16-May-2026

TTRPG? DSL?

Wait, what?

- A Domain-Specific Language (DSL) can be **very** hard to design.
- A Table-Top Role-Playing Game (TTRPG) is a complicated problem domain. (And fun. And low-risk.)
- Designing a DSL for a TTRPG can provide insight into DSL design using Python.

We'll look at two examples.

DSL's Can Be Complicated

Important

Python may be all you need.

Start there.

Avoid writing a parser.

Dice-Rolling Language

3d6+2

This is an easy place to start.

`3d6 + 2` is a syntax for rolling polydedral dice. In this case 3 6-sided die; then adding 2.

D&D used platonic solids: d4, d6, d8, d12, d20.
Other games added a d10 (which isn't a platonic solid)

We *can* parse with a regular expression. Or...

In Python


```
weapon_damage = 3 * D6 + 2

characteristic = (4 * D6).keep(3)
character = [
    characteristic.roll() for _ in range(6)
]
```

Looks good to me. `3*D6+2` vs. `3d6+2`

Cost: one extra `*`.

How does that work?


```
class Die:
    def __init__(self, d: int) -> None:
        ...
    def __rmul__(self, n: int) -> "Die":
        ...
    def __add__(self, n: int) -> "Die":
        ...
    def keep(self, k: int) -> "Die":
        ...
D6 = Die(6)
```

The implementation isn't *too* complicated.

OpenD6 Magic Spell Language

The OpenD6 Fantasy rules define a bunch of magical spells.

There are many pages of rules.

Some aids for doing spell definitions:

- An "Adjusting and Readjusting" admonition.
- A "Spell Design" sheet.
- *These words*: "A calculator might also help." *Really.*

```
Spell(
    name="Push",
    skill="Apportation",
    notes="Even the most subtle of things...",
    effect=TimeEffect("sends a small object into the future", "10 min"),
    duration=DurationAspect(measure="10 minutes"),
    range=RangeAspect(measure="touch"),
    casting_time=CastingTimeAspect(measure="1 round"),
    speed=SpeedAspect.based_on(("range",), ""),
    other_aspects={
        "gestures": GesturesAspect("Wave one hand ...", "simple"),
        "incantations": IncantationsAspect(
           "Where did it go?", "sentence"
        ),
    },
    other_conditions=[
        ArcaneKnowledgeAspect(0, "time"),
    ],
)
```

That looks acceptable

Most of the spell details look like this.

```
Spell(
    ...
    duration=DurationAspect(measure="10 minutes"),
    ...
),
```

The *label* = *value* is similary to TOML, JSON, etc.

We can use `2*D` as a formal specification for damage and skills.

Summary

Don't invent a DSL from scratch.

Start with Python object definitions as your declarative language.

Don't write a parser unless there's no alternative.

Link to GitHub Repository

https://github.com/slott56/writing-tools